

- Colin Wei, Yining Chen, and Tengyu Ma. Statistically meaningful approximation: a case study on approximating turing machines with transformers. *CoRR*, abs/2107.13163, 2021. URL <https://arxiv.org/abs/2107.13163>.
- Colin Wei, Sang Michael Xie, and Tengyu Ma. Why do pretrained language models help in downstream tasks? an analysis of head and prompt tuning, 2022.
- Masami Yamasaki. The lower bound of the capacity for a neural network with multiple hidden layers. In *ICANN'93: Proceedings of the International Conference on Artificial Neural Networks Amsterdam, The Netherlands 13–16 September 1993* 3, pages 546–549. Springer, 1993.
- Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J Reddi, and Sanjiv Kumar. Are transformers universal approximators of sequence-to-sequence functions? *arXiv preprint arXiv:1912.10077*, 2019a.
- Chulhee Yun, Suvrit Sra, and Ali Jadbabaie. Small relu networks are powerful memorizers: a tight analysis of memorization capacity. *Advances in Neural Information Processing Systems*, 32, 2019b.
- Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank Reddi, and Sanjiv Kumar. Are transformers universal approximators of sequence-to-sequence functions? In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=ByxRM0Ntvr>.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64(3):107–115, 2021.
- Jingzhao Zhang, Sai Praneeth Karimireddy, Andreas Veit, Seungyeon Kim, Sashank J Reddi, Sanjiv Kumar, and Suvrit Sra. Why are adaptive methods good for attention models?, 2020.

A Experimental Details

All the experiments are run on a NVIDIA RTX A6000 GPU. For experiments with Llama 7B model, we use batch size 32 and learning rate 0.001. For experiment on WMT14 En-Fr translation, we only compute the loss on the first 100 examples for computational efficiency.

We use Adam optimizer and optimal learning rate from grid search at 0.1 for prompt-tuning and at 0.001 for fine-tuning in Section 7.2.

In Section 7.3, we use the default loss function in Huggingface implementation for causal language models. We use prompt length $m = 10$ and the prompt tokens are initialized as the first m tokens in the model vocabulary.

B Additional Experiments

As mentioned in Section 7.3, the second real world dataset used in our experiment is WMT14 En-Fr translation in order to illustrate that the spectral norm of soft prompts increases during training. We show the curve in Figure B.

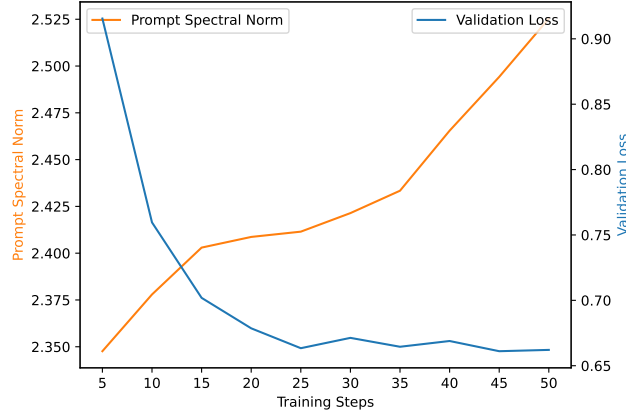


Figure 3: Increasing prompt spectral norm during tuning on WMT14 En-Fr translation dataset.

C Proof of Lemmas and Theorems

C.1 Proof of Lemma 1

For the sequence-to-sequence function space $\mathcal{F}_L$ with functions $f : [0, 1]^{d \times m} \rightarrow [0, 1]^{d \times m}$, we can build a sequence-to-sequence function $\bar{g} : [0, 1]^{d \times (m_p + m)} \rightarrow [0, 1]^{d \times (m_p + m)}$ such that for any $f \in \mathcal{F}_L$, we can find $\mathbf{P} \in \mathbb{R}^{d \times m_p}$, $d_p(\bar{g}([\mathbf{P}, \cdot]), f) \leq \epsilon/2$.

Proof. we first quantize the input and output sequence space of $[0, 1]^{d \times m}$ into a grid space $G_{\delta, m} = \{0, \delta, 2\delta, \dots, 1 - \delta\}^{d \times m}$, which leads to $C = (\frac{1}{\delta})^{\frac{1}{d \times m}}$ functions considering all input and output mappings in this grid. We index these C functions as $\bar{F}_L = \{\bar{f}_1, \bar{f}_2, \dots, \bar{f}_C\}$. For $\mathbf{X} \notin G_{\delta, m}$, we let $\bar{f}_i(\mathbf{X}) = \bar{f}_i(\mathbf{X}^*)$ if $k_{i,j}\delta < \mathbf{X}_{i,j} \leq (k_{i,j} + 1)\delta$ and $\mathbf{X}^* \in G_{\delta, m}$.

Then for any $f \in \mathcal{F}_L$, we can find a function $\bar{f} \in \bar{F}_L$ such that $d_p(\bar{f}, f) = (\int \|\bar{f}(\mathbf{X}) - f(\mathbf{X})\|_p^p d\mathbf{X})^{1/p} \leq (\int L^p m d\delta^p d\mathbf{X})^{1/p} = L(md)^{\frac{1}{p}}\delta$. We choose $\delta = \delta_1$ such that $L(md)^{\frac{1}{p}}\delta \leq \epsilon/2$. For the prompt part, we choose m_p such that $\frac{1}{\delta^{d \times m_p}} \geq C$. Then we can build a set of quantized prompts in $G_{\delta, m_p} = \{0, \delta, 2\delta, \dots, 1 - \delta\}^{d \times m_p}$ to index these C functions. We denote this set of prompts as $\{\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_C\}$. Finally we can create the quantized function $\bar{g}$ and let

$\bar{g}([\mathbf{P}_i, \mathbf{X}])_{:,m_p} = \bar{f}_i(\mathbf{X})$ and $\bar{g}([\mathbf{P}_i, \mathbf{X}])_{:,m_p} = 0, \forall \mathbf{X} \in [0, 1]^{d \times m}, \mathbf{P} \in G_{\delta, m_p}$. For $\mathbf{P} \notin G_{\delta, m_p}$, we set $\bar{g}([\mathbf{P}, \mathbf{X}]) = \bar{g}([\mathbf{P}^*, \mathbf{X}])$ if $k_{i,j}\delta < \mathbf{P}_{i,j}, \mathbf{P}_{i,j}^* \leq (k_{i,j} + 1)\delta$ and $\mathbf{P}^* \in G_{\delta, m_p}$.

Therefore, with a properly chosen $\delta = \delta_1$, for any $f \in \mathcal{F}_L$, we can find $\mathbf{P} \in \mathbb{R}^{d \times m_p}$ such that $d_P(f, \bar{g}([\mathbf{P}, \cdot])_{:,m_p}) = d_p(\bar{f}, f) \leq \epsilon/2$.

□

C.2 Proof of Lemma 2

For any given quantized function $\bar{f} : [0, 1]^{d \times m} \rightarrow [0, 1]^{d \times m}$ with quantization at interval δ , $\exists \bar{h} \in$

$$\bar{\mathcal{T}}^{2,1,1} \text{ such that } \bar{f} = \bar{h} \text{ with positional embedding } \mathbf{E} = \begin{bmatrix} 0 & 1 & 2 & \dots & m-1 \\ 0 & 1 & 2 & \dots & m-1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 1 & 2 & \dots & m-1 \end{bmatrix}.$$

Proof. The proof is given following Section C in Yun et al. [2019a] appendix. With Section C.1 in Yun et al. [2019a], there exists a function g_q composed of $\frac{dm}{\delta}$ token-wise feed-forward layers with hidden layer size $r = 1$ and ReLU activation to implement this scalar quantization on each input element:

$$g_q^{ent}(t) = \begin{cases} k\delta & \text{if } k\delta \leq t < (k+1)\delta, k = 0, 1, \dots, m/\delta - 1 \\ -\delta^{-md} & \text{otherwise} \end{cases}$$

Then with Section C.2 in Yun et al. [2019a], we can stack $m(1/\delta)^d + 1$ attention layers to map all possible input sequences in grid $[0 : \delta : 1 - \delta]^d \times [1 : \delta : 2 - \delta]^d \times \dots \times [m-1 : \delta : m - \delta]^d$ to distinct numbers which are at least δ from each other.

Finally we only require $O(m(1/\delta)^{dm})$ layers to map these distinct numbers to expected outputs. □

C.3 Proof of Lemma 3

Lemma 3 is almost the same as [Yun et al., 2019a] except that we use $\epsilon/2$ instead of $\epsilon/3$.

C.4 Extension of Theorem 1 to Next-token Predictors

As an extension of Theorem 1, we consider approximating a set of sequence-to-sequence functions when we use a transformer layer as a next-token predictor. We consider a set of sequence-to-sequence functions F_L with Lipschitz constant L under norm p . $f \in F_L : [0, 1]^{d \times m_1} \rightarrow [0, 1]^{d \times m_2}$ accepts an input of length m_1 and outputs a sequence of length m_2 . For any $\mathbf{X}, \mathbf{X}' \in [0, 1]^{d \times m_1}$, we have $\|f(\mathbf{X}) - f(\mathbf{X}')\|_p \leq L\|\mathbf{X} - \mathbf{X}'\|_p$.

Next we show that we can construct a transformer τ which can approximate any $f \in F_L$ with prompt-tuning when we use it as a next-token predictor.

Theorem 5. For any $f \in F_L$, we can construct a transformer τ such that for any $f \in F_L, 1 \leq p < \infty$ and $\epsilon > 0$, we can find a prompt $\mathbf{P} \in [0, 1]^{d \times m_p}$, such that $d_p(f, h(\mathbf{P})) \leq \epsilon$, where

$$h(\mathbf{P}) = \tau_1([\mathbf{P}, \cdot])_{:, -1} \times \tau_2([\mathbf{P}, \cdot])_{:, -1} \times \dots \times \tau_{m_2}([\mathbf{P}, \cdot])_{:, -1}.$$

τ_i is the sequence-to-sequence function implemented with the transformer τ when accepting sequences with length $m_p + m_1 + i$.

Proof. Similar to Theorem 1, we quantize the inputs to grid of $[0 : \delta : 1 - \delta]$ with interval δ and set $m_p = (\delta^{d \times m_2})^{\delta^{d \times m_1}}$. δ is chosen such that $L(m_1 d)^{1/p} \delta \leq m_2^{-1/p} \epsilon$. We index the $C = (\delta^{d \times m_2})^{\delta^{d \times m_1}}$ different f s as $f^1, f^2, \dots, f^C$ and its sub-function to generate the i -th output token as f_i^j . The C sequence-to-sequence functions can then be indexed by C distinct prompts. Similar to Lemma 2, we can construct a transformer which can map all possible input sequences in grids $[0 : \delta : 1 - \delta] \times \dots \times [m-1 : \delta : m - \delta]^d, 0 \leq m \leq m_1 + m_2 + m_p - 1$ to distinct numbers. A final series of MLP layers then map these distinct numbers to desired output vectors where inputs